

```

class MeansEndAnalysis:

    def __init__(self, operators):
        self.operators = operators

    def solve(self, current, goal):
        print(f'Current State: {current} | Goal State: {goal}')

        # Goal reached
        if current == goal:
            return []

        # Find the difference
        diff = self.find_difference(current, goal)

        if not diff:
            return []

        # Select an operator that can reduce the difference
        op = self.select_operation(diff)

        if not op:
            print(f'No operation found to reduce the difference: {diff}')
            return None

        # Check if preconditions are satisfied
        for key, value in op['precond'].items():
            if current.get(key) != value:
                print(f'Precondition {key}={value} not satisfied.')

```

```
return None
```

```
# Apply operator
```

```
new_state = current.copy()
```

```
new_state.update(op['effect'])
```

```
print(f'Applying operation: {op['name']}')  
print(f'New State: {new_state}')
```

```
# Recursively solve from the new state
```

```
remaining_path = self.solve(new_state, goal)
```

```
if remaining_path is None:
```

```
    return None
```

```
return [op['name']] + remaining_path
```

```
def find_difference(self, current, goal):
```

```
    for key in goal:
```

```
        if current.get(key) != goal.get(key):
```

```
            return (key, goal.get(key))
```

```
    return None
```

```
def select_operation(self, diff):
```

```
    key, value = diff
```

```
    for op in self.operators:
```

```
        if op['effect'].get(key) == value:
```

```
    return op
```

```
    return None
```

```
if __name__ == "__main__":
```

```
    operators = [
```

```
        {
```

```
            'name': 'drive_to_work',
```

```
            'precond': {
```

```
                'has_car': True,
```

```
                'at_home': True
```

```
            },
```

```
            'effect': {
```

```
                'at_work': True
```

```
            }
```

```
        }
```

```
    ]
```

```
    current_state = {
```

```
        'has_car': True,
```

```
        'at_home': True
```

```
    }
```

```
    goal_state = {
```

```
        'at_work': True
```

```
    }
```

```
mea = MeansEndAnalysis(operators)

plan = mea.solve(current_state, goal_state)


print("\nExecuted Plan:", plan)

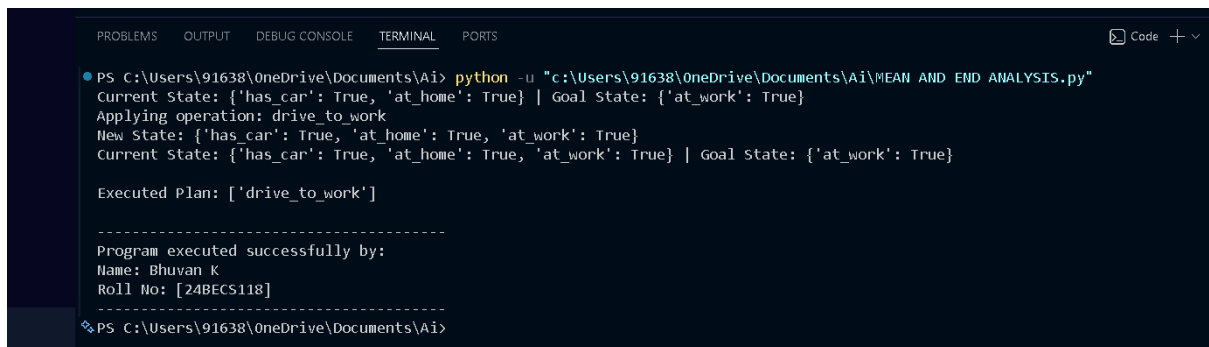
print("\n-----")

print("Program executed successfully by:")

print("Name: Bhuvan K")

print("Roll No: [24BECS118]")

print("-----")
```



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
PS C:\Users\91638\OneDrive\Documents\Ai> python -u "c:\Users\91638\OneDrive\Documents\Ai\MEAN AND END ANALYSIS.py"
Current State: {'has_car': True, 'at_home': True} | Goal State: {'at_work': True}
Applying operation: drive_to_work
New State: {'has_car': True, 'at_home': True, 'at_work': True}
Current State: {'has_car': True, 'at_home': True, 'at_work': True} | Goal State: {'at_work': True}

Executed Plan: ['drive_to_work']

-----
Program executed successfully by:
Name: Bhuvan K
Roll No: [24BECS118]
-----
PS C:\Users\91638\OneDrive\Documents\Ai>
```